Parallel Programming for Machine Learning
Final-term assignment: *Mean shift clustering*

Francesco Acciaioli
University of Florence
francesco.acciaioli@stud.unifi.it

Abstract

Mean shift clusters data by moving every point towards a mode of a kernel density estimate. Applied to the colours of an image, it gives a segmentation without choosing the number of clusters in advance, but each iteration costs $O(N^2)$ in the number of pixels. I implemented it in C++17 in four variants: a scalar sequential baseline, a single-thread SIMD version, an OpenMP version with a vectorized inner loop, and an Array-of-Structures copy of the latter to study the memory layout. Since every point converges on its own, the parallel version needs no locks, only reductions. On images from the Berkeley Segmentation Dataset and an Apple M2 (4 performance and 4 efficiency cores), vectorization alone gives $1.84\times$, and with 8 threads a 128×128 image goes from 13.15 s to 1.51 s ($8.7\times$). With a large bandwidth the speedup reaches $15.2\times$, mostly because the scalar baseline gets slower. Threads scale almost linearly on the 4 performance cores; the efficiency cores add much less.

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

This report describes a sequential and a parallel implementation of mean shift clustering for colour image segmentation, and measures how much the parallel one gains and why. Code, benchmark and results are in the repository https://github.com/FranciAccia/MeanShiftParallel.

1.1. The problem

Mean shift looks for the local maxima (*modes*) of a density estimated from the data with a kernel (KDE). Each point is moved uphill on this density until it stops at a mode, and the points that stop at the same mode form a cluster. The number of clusters is not an input, as it is in K-means. The only parameter is the bandwidth h of the kernel, which decides how far apart two structures must be to count as different (Figure 1).

For segmentation, each pixel is a point in RGB space. Replacing every pixel with the colour of its mode gives an image made of a few flat colours (Figure 2).

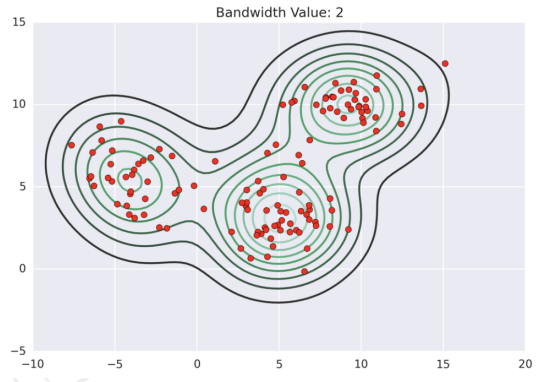


Figure 1: Kernel density estimate of a 2D point set. The modes (centres of the contour lines) are the cluster centres that mean shift finds. Figure from the course slides [1].

1.2. Why it is slow

At every iteration, each point has to compute the kernel against all N points, so one iteration over the whole image costs $O(N^2)$, and points need about 43 iterations to converge in my main configuration. A 128×128 image ($N = 16\,384$) already means about $1.1 \cdot 10^{10}$ kernel evalua-

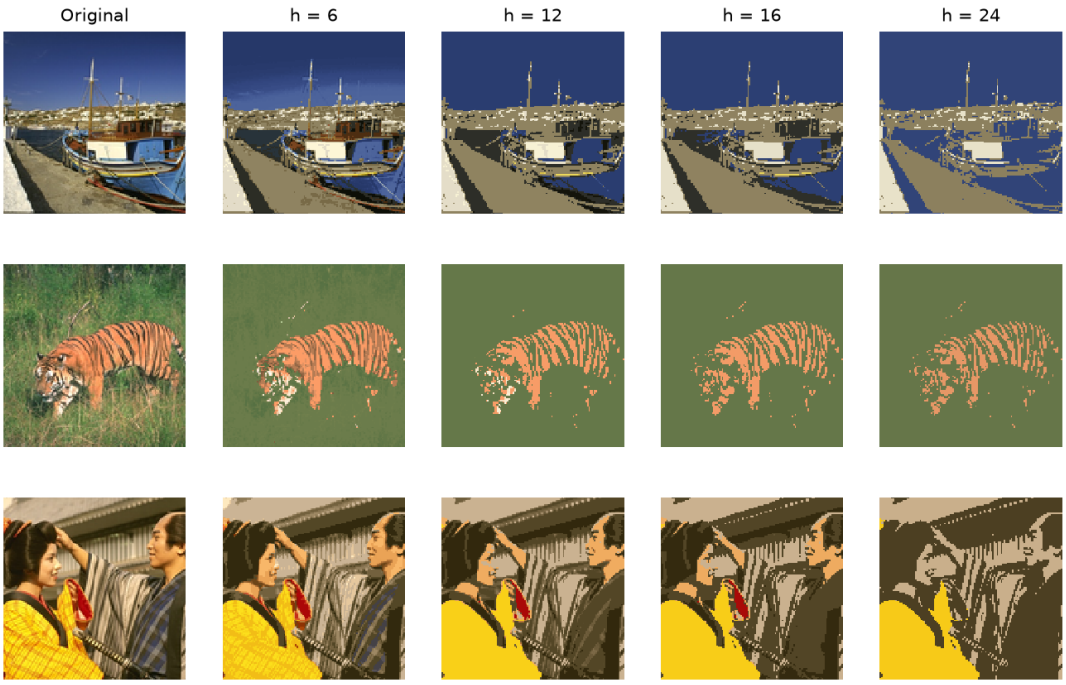


Figure 2: Segmentation of the three BSDS500 images (128×128) for increasing bandwidth. With a small h many shades survive; with a large one, different objects get the same colour.

tions, 13 s on one core. A full-size BSDS image (481×321) would take hours.

The good news is that the trajectories do not interact: a point only reads the original dataset, which never changes. This makes the problem easy to split among threads.

1.3. Goals

I wrote a scalar sequential version as the reference for both results and times, then a parallel version that uses OpenMP threads over the points and SIMD instructions over the neighbours. Finally I measured speedup and efficiency for different numbers of threads, and the effect of the bandwidth, of the OpenMP schedule and of the memory layout.

2. The algorithm

2.1. Sequential version

The pixels are points $x_1, \dots, x_N \in \mathbb{R}^3$ with values in $[0, 255]$. With a Gaussian kernel of bandwidth h, one step moves a point y to the weighted mean of the dataset:

$$y \leftarrow \frac{\sum_j w_j\, x_j}{\sum_j w_j}, \qquad w_j = \exp\!\left(-\frac{\|y - x_j\|^2}{2h^2}\right). \tag{1}$$

This is the standard (non-blurring) mean shift: the dataset stays fixed and only the copy y of each point moves. Neighbours farther than $3h$ would get a weight below 0.012, so I set it to zero. A point stops when it moves less than εh ($\varepsilon = 10^{-3}$) or after 100 iterations.

The program has three phases. The first is the shift of Algorithm 1, which costs $O(N^2 \cdot \text{iter})$. The second finds the clusters: it scans the modes in index order and opens a new cluster for every mode that is at least $h/2$ away from the centres found so far ($O(NK)$ for K clusters). The third gives each pixel the label of the centre closest to its mode. Phases 2 and 3 together take about 0.2 ms, against 7–13 s for phase 1, so phase 1 is the only part worth optimizing.

Algorithm 1 Shift phase of mean shift

for $i = 1 \dots N$ **do** ▷ independent points
  $y \leftarrow x_i$
  repeat
    $s \leftarrow 0,\ s_w \leftarrow 0$
    for $j = 1 \dots N$ **do** ▷ all the neighbours
      $d^2 \leftarrow \|y - x_j\|^2$
      if $d^2 \le (3h)^2$ **then**
        $w \leftarrow \exp(-d^2/2h^2)$
        $s \leftarrow s + w\,x_j;\ s_w \leftarrow s_w + w$
      end if
    end for
    $y_{old} \leftarrow y;\ y \leftarrow s/s_w$
  until $\|y - y_{old}\| < \varepsilon h$ **or** 100 iterations
  $mode_i \leftarrow y$
end for

2.2. What can run in parallel

The outer loop is the natural place for threads: iteration i reads the dataset and writes only the mode of point i. Its iterations do not all cost the same, though, because some points need many more steps than others, so the choice of schedule matters.

The inner loop does the same arithmetic on every neighbour and accumulates four sums (s_r, s_g, s_b, s_w). That is a reduction, and it can be vectorized.

The labelling is also independent per pixel, so I parallelized it too. I kept the cluster search sequential on purpose: scanning the modes in a fixed order makes the clusters the same for any number of threads.

3. Parallel implementation

3.1. Variants

All four variants use the same parameters and the same kernel. They only differ in how they run phase 1 (Table 1).

3.2. Threads (OpenMP)

One directive splits the points among the threads:

```
#pragma omp parallel for \
        schedule(runtime) reduction(+ : total)
for (int i = 0; i < n; ++i) {
    Pixel y = d.aos[i];
    total += converge_simd_soa(R, G, B, n, y,
                               k, max_iter);
    modes[i] = y;
}
```

Variant	Parallelism	Layout
`seq`	none, scalar (baseline)	AoS
`simd`	1 thread, `omp simd` inner loop	SoA
`par`	`omp parallel for` + SIMD	SoA
`par_aos`	as `par` (layout study)	AoS

Table 1: Implemented variants.

With `schedule(runtime)` the benchmark can set the policy and chunk size through `omp_set_schedule`, so testing a new schedule does not require recompiling. All the experiments use `dynamic,16` unless they say otherwise (Section 5.5 explains why).

3.3. Synchronization

The parallel loop needs no locks, `critical` sections or atomics. Nobody writes the dataset, and iteration i writes `modes[i]` once, when point i has converged. The only value that all threads update is the iteration counter I keep for statistics, and `reduction(+:total)` gives each thread its own copy and adds them up at the end. Two threads can write to the same cache line of `modes` only at the border between two chunks, once per point, after about $43 \cdot N$ kernel evaluations. That is too rare to matter, so I did not add padding.

3.4. Vectorization (SIMD)

The inner loop carries `#pragma omp simd reduction(+:sr,sg,sb,sw)`. The M2 has 128-bit NEON registers, so one instruction handles 4 neighbours in single precision. The loop did not vectorize well as written in Algorithm 1, and it took two changes.

First, the cutoff became a select instead of an `if`: `w = (d2 <= cutoff2) ? exp(...) : 0`. All lanes now run the same instructions, and the far neighbours get weight zero.

Second, I replaced `std::exp`. Inside the vector loop the compiler turned it into one library call

per lane, and the SIMD version ended up no faster than the scalar one. My `fast_exp` computes 2^k by writing k into the IEEE-754 exponent and multiplies it by a degree-5 polynomial; the relative error is below $4 \cdot 10^{-7}$ and it is inline and branch-free. The scalar baseline uses the same function, so the difference between `seq` and `simd` comes only from vectorization.

The baseline keeps the `if`, so it skips the exponential for far neighbours, and I disabled its vectorization explicitly with `#pragma clang loop vectorize(disable)`. The clang report (`-Rpass=loop-vectorize`) shows the inner loops of `simd`, `par` and `par_aos` vectorized with width 4 and interleave 2, and the `seq` loop left scalar. I did not use `-ffast-math`: the compiler reorders floating-point sums only where `omp simd reduction` allows it.

3.5. Data layout

The `Dataset` structure holds the pixels twice, built when the image is loaded and outside the timed code. One copy is an Array of Structures (`struct Pixel {float r,g,b;}`), the other a Structure of Arrays with three separate arrays `r`, `g`, `b`. With SoA, one vector load brings in 4 values of the same channel. With AoS the loop reads with stride 3 and has to separate the channels first. I used `float` because colours in $[0, 255]$ do not need double precision, and 4 lanes are better than 2.

3.6. Problems I ran into

The first SIMD version was no faster than the scalar code; the culprit was `std::exp`, as described above. The default static schedule turned out to be the slowest one, because points in textured areas need more iterations than points in flat areas (Section 5.5). The laptop has no fan, and under continuous load the times grew by up to 50%, so I had to add a pause between runs (Section 4).

4. Experimental setup

4.1. Hardware and software

- **CPU**: Apple M2 (MacBook Air, Mac14,2), 8 cores: 4 performance (P, up to 3.5 GHz, 128 KB L1d each, 16 MB shared L2) and 4 efficiency (E, up to 2.4 GHz, 4 MB shared L2); 128-bit ARM NEON SIMD; 8 GB unified memory. Passive cooling (no fan).
- **Software**: macOS 26.6, Apple clang 21.0.0, LLVM OpenMP runtime (`libomp`) 21.1.8 from Homebrew, CMake 4.2.
- **Flags**: `-std=c++17 -O3 -march=native -Xpreprocessor -fopenmp` (Release build).

4.2. Dataset and preprocessing

I took three images from the BSDS500 test set [3] with different colour content: 118015 (boats in a harbour, used as the main image), 108004 (a tiger in the grass) and 145059 (people in coloured clothes). A Python script crops the central square of each image and resizes it to 64^2, 96^2 and 128^2 pixels with a Lanczos filter. The images have to be this small because of the $O(N^2)$ cost: 128^2 is the smallest size whose sequential time is above the 10 s asked by the guidelines (13.15 s with $h = 12$). Preprocessing runs once, offline, and is not timed. The program reads the images with `stb_image`.

4.3. Parameters

The main configuration is image 118015 at 128×128 ($N = 16\,384$), $h = 12$, 8 threads and `dynamic,16`. Each experiment changes one thing:

- **strong scaling**: 1, 2, 4, 6, 8, 12 and 16 threads (up to twice the cores) on all three sizes;
- **bandwidth**: $h \in \{6, 8, 12, 16, 24, 32\}$, on 96×96 so that the sequential runs stay short;
- **schedule**: `static`, `static,1`, `static,64`, `dynamic` with chunk 1, 16 and 256, `guided`;

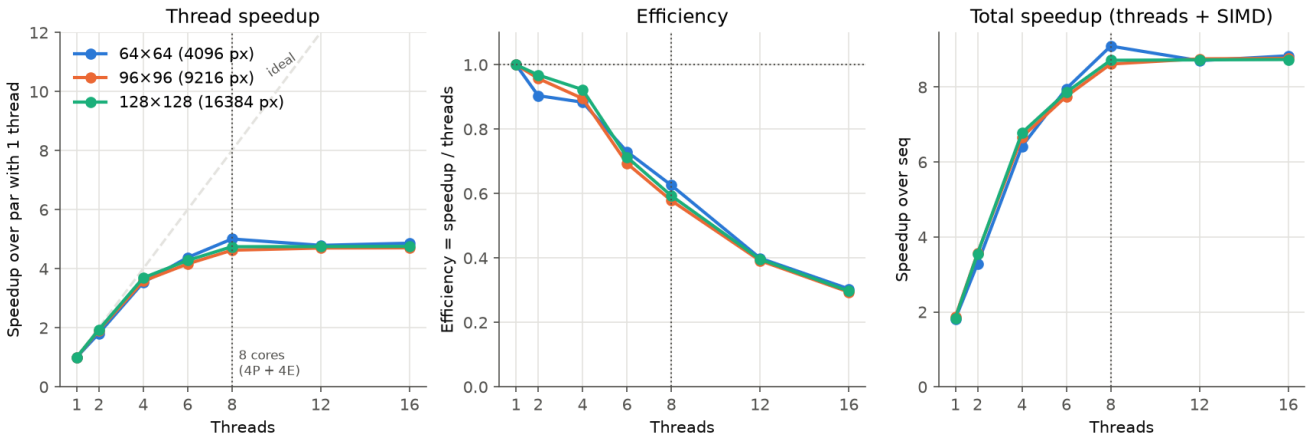


Figure 3: Strong scaling for the three image sizes. Left: speedup of the threads over `par` with 1 thread (dashed line: ideal). Centre: efficiency. Right: total speedup over the scalar sequential baseline. The dotted line marks the 8 cores of the M2.

- **layout**: AoS and SoA, with 1 and 8 threads.

4.4. How I measured

Each configuration runs once as a warm-up, which is discarded, and then 5 times. I report mean, standard deviation, minimum, maximum and a 95% confidence interval ($1.96\,\sigma/\sqrt{5}$). Wall-clock time comes from `omp_get_wtime()` and covers only the clustering, not the image loading or the layout conversion. CPU time, summed over all threads (user + system), comes from `getrusage`. Speedup is $S(p) = T_{seq}/T_{par}(p)$ and efficiency $E(p) = S(p)/p$.

Since the machine has no fan, after every run the benchmark waits as long as the run took (5 s at most), so that each run starts at about the same temperature. On macOS I could not fix the CPU frequency or pin threads to cores. I closed the other applications during the measurements.

5. Results

5.1. Correctness

Before any benchmark, the program runs every variant on the three 64×64 images and compares it with `seq`. If the check fails, the benchmark stops. A variant passes if it finds the same number of clusters, gives every pixel the same label, and has every mode within 1% of h (0.12 on the 0–255 scale) of the sequential one.

All variants pass. Clusters and labels match exactly (9, 5 and 14 clusters; 4096 of 4096 labels), and the largest mode difference is $1.1\cdot10^{-2}$, $6.7\cdot10^{-2}$ and $7.5\cdot10^{-2}$ on the three images. Some difference is expected: the SIMD reduction adds the terms in a different order, and the εh stopping rule can then stop two trajectories at slightly different points. `simd`, `par` and `par_aos` give exactly the same modes as each other for any number of threads, since each point is still computed by one thread in a fixed order.

5.2. Vectorization

On one thread `simd` is 1.83–1.86× faster than `seq`, at every image size (Figure 5). NEON has 4 lanes, so why not 4×? The two loops do not do the same work. The scalar loop skips the exponential for every neighbour beyond $3h$, while the vector loop computes it for all neighbours and throws the far ones away with the select. The bandwidth experiment in Section 5.4 confirms this.

5.3. Strong scaling

Table 2 and Figure 3 show the strong scaling on the main image. I report two speedups: the one due to threads alone, $T_{par}(1)/T_{par}(p)$, and the total one over the scalar baseline, which includes vectorization.

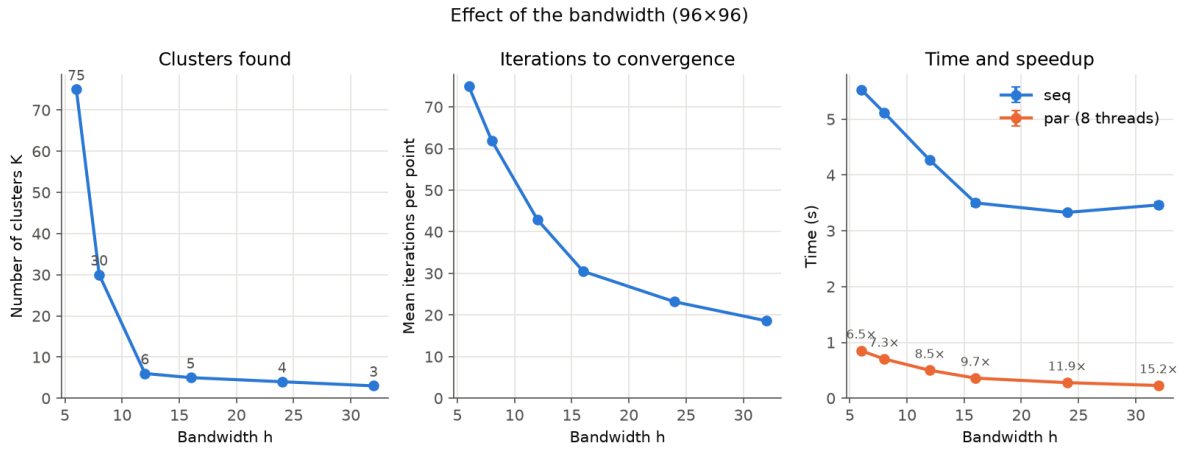


Figure 4: Effect of the bandwidth on the 96×96 image: number of clusters, mean iterations per point, and time of `seq` and `par` (8 threads) with the resulting speedup.

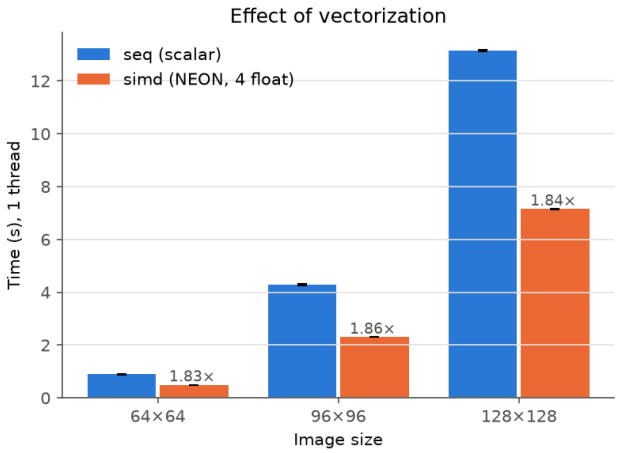


Figure 5: Scalar baseline and SIMD version on one thread, with 95% confidence intervals and the speedup of `simd` over `seq`.

p	Time (s)	S_{thr}	E	S_{tot}	CPU (s)
seq	13.151 ± 0.022	–	–	1.00	13.14
1	7.159 ± 0.005	1.00	1.00	1.84	7.16
2	3.697 ± 0.005	1.94	0.97	3.56	7.39
4	1.940 ± 0.014	3.69	0.92	6.78	7.73
6	1.673 ± 0.002	4.28	0.71	7.86	9.93
8	1.509 ± 0.007	4.75	0.59	**8.72**	11.29
12	1.507 ± 0.003	4.75	0.40	8.73	11.35
16	1.505 ± 0.001	4.76	0.30	8.74	11.39

Table 2: Strong scaling of `par` on the 128×128 image, $h = 12$ (mean $\pm$ 95% CI). S_{thr} and E: speedup and efficiency over `par` with 1 thread; S_{tot}: speedup over the scalar `seq`.

Up to 4 threads the scaling is close to ideal: efficiency is 0.97 with 2 threads and 0.92 with 4. These threads run on the performance cores.

From 4 to 8 threads the new threads land on the efficiency cores, and the speedup only goes from 3.69 to 4.75. Four E-cores add about as much as one P-core, so on this loop an E-core is worth roughly 0.26 of a P-core. Measured in P-core equivalents the machine offers about $4 + 4 \cdot 0.26 \approx 5$, and 4.75 is 94% of that. The CPU time tells the same story (Figure 6): it grows from 7.7 s to 11.3 s because the same work takes longer on the slower cores.

With 12 and 16 threads there are no more cores to use. The speedup stays at $4.75\times$, and oversubscription costs nothing I could measure.

The three image sizes give nearly identical curves. Even at 64×64 each thread gets hundreds of points, and each point costs about $43 \cdot N$ kernel evaluations, so the OpenMP overhead never shows up. The best result is with 8 threads: $8.72\times$ over the baseline (13.15 s to 1.51 s), which is $1.84\times$ from SIMD times $4.75\times$ from threads.

5.4. Bandwidth

Table 3 and Figure 4 show what happens on the 96×96 image as h grows. The density becomes smoother: the clusters drop from 75 to 3, and the mean number of iterations from 75 to 19, so both

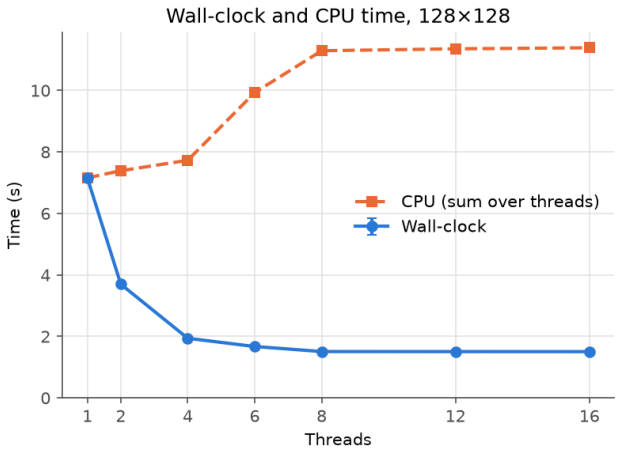


Figure 6: Wall-clock time and CPU time (sum over threads) on the 128×128 image.

versions run faster. Figure 2 shows the visual side. Around $h = 12$ the main objects are separated, while at $h = 24$ the boat already takes the colour of the sea.

h	K	iter	seq (s)	par (s)	S_{tot}
6	75	74.9	5.530	0.850	6.5×
8	30	61.9	5.118	0.704	7.3×
12	6	42.9	4.265	0.499	8.5×
16	5	30.5	3.499	0.360	9.7×
24	4	23.2	3.329	0.279	11.9×
32	3	18.6	3.463	0.228	15.2×

Table 3: Effect of the bandwidth (96×96, `par` with 8 threads). K: clusters; iter: mean iterations per point.

The speedup also grows with h, from $6.5\times$ to $15.2\times$. Dividing each time by the number of iterations shows why. One iteration of `par` costs the same at every bandwidth (11.3 ms at $h = 6$, 12.3 ms at $h = 32$), because the vector loop always processes every neighbour. One iteration of `seq` goes from 74 ms to 186 ms: with a wider kernel, fewer neighbours fall outside the cutoff, and the scalar loop can skip fewer exponentials. So the values above $8\times$ do not mean the threads scale better than linearly. The baseline simply gets more expensive as h grows, while the vector loop does not. By the same logic, the SIMD gain is smaller at small h.

5.5. Schedule

Figure 7 compares the OpenMP schedules with 8 threads. Plain `static` is the slowest at 2.24 s, 42% more than the best. It gives each thread one block of $N/8$ consecutive pixels, that is, one horizontal band of the image. How many iterations a pixel needs depends on where it is (the flat sky converges fast, the textured harbour slowly), so some threads finish long before the others. `static,1`, which deals the pixels out one at a time, already runs in 1.60 s, as fast as the dynamic schedules. The imbalance therefore comes from the spatial grouping of the work, not from static scheduling itself. All the schedules except plain `static` are within 5% of each other (1.58–1.65 s), and `dynamic,1` has the largest variance. I kept `dynamic,16`, which balances the load without handing out too many small chunks.

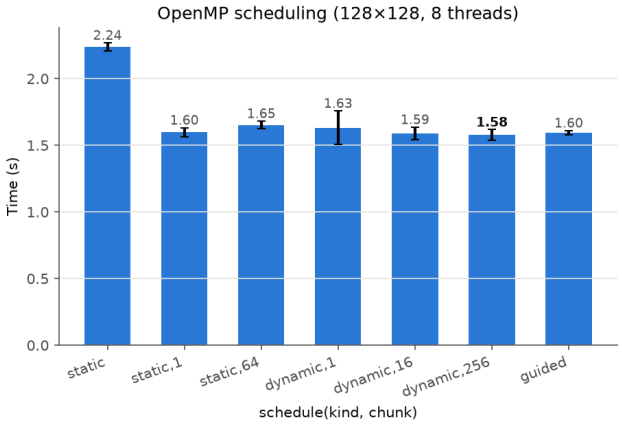


Figure 7: Time with different OpenMP scheduling policies (128×128, 8 threads, 95% CI). The best one is in bold.

5.6. Memory layout

With the same vectorized loop, SoA beats AoS by 7.7% on one thread (7.18 against 7.73 s) and by 8.3% on 8 threads (1.51 against 1.64 s), as Figure 8 shows. The gap is small because the data is small: one copy of the image takes $16\,384 \times 12$ B $= 192$ KB and stays in the L2 cache. The loop is limited by arithmetic, not by memory, and the price of AoS is the extra work to separate the channels of the stride-3 loads.

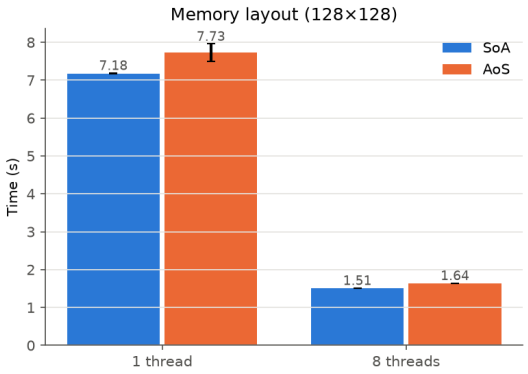


Figure 8: SoA and AoS layouts (128×128, `par`, 95% CI).

5.7. Where the time goes

The program is compute-bound. The dataset fits in cache and is only read; one kernel evaluation takes about 0.63 ns on one P-core with SIMD (7.16 s for $1.14 \cdot 10^{10}$ evaluations), and most of the arithmetic in the loop is the exponential. Beyond 4 threads the limit is the hardware: the extra cores are the slow ones. Inside each thread, the branch-free cutoff wastes work on far neighbours, which is why SIMD gives $1.84\times$ and not more at $h = 12$.

6. Discussion

Amdahl's law. The sequential part of the parallel program is the cluster search: 0.15 ms out of 7.16 s, a fraction $f \approx 2 \cdot 10^{-5}$. With such a small f, Amdahl's law [2] would allow a speedup close to p on any realistic number of cores, so the sequential code is not what stops the scaling. The cores are: they are not all equal, and the ideal speedup on this machine is about 5, not 8.

Gustafson's law. The work per point grows with N, so larger images make the parallel part even more dominant. My sizes already show it: efficiency is the same from 64^2 to 128^2, because the overhead is negligible even on the smallest image. I did not run a weak-scaling test: with $O(N^2)$ work, keeping the work per thread constant means multiplying N by $\sqrt{p}$, so 16 threads would need a 256×256 image with 16 times the work of the 128×128 one. Its sequential reference alone would take about 20 minutes with my protocol (6 runs of $\approx$ 210 s), on a machine that has only 8 cores anyway.

Overhead. Creating the threads and handing out chunks takes microseconds, while every run lasts at least 0.1 s. Going from 8 to 16 threads does not change the time. The extra CPU time at 8 threads comes from the slower E-cores, not from OpenMP.

Limitations. The laptop has no fan. At 8 threads under continuous load, one run goes from 1.55 to 1.85 s within 20 s and gets about 50% slower after 2 minutes. The pause between runs reduces the effect, but the absolute times still depend on temperature. macOS also does not let me fix the CPU frequency or choose which cores the threads run on; the operating system decides the placement on P and E cores. Finally, the $O(N^2)$ cost keeps the images at 128^2 pixels or less. Full-resolution images would need a faster way to find the neighbours.

What I learned. A loop only vectorizes well if its whole body is inline and free of branches: a single library call, `std::exp`, was enough to cancel the SIMD gain. The baseline must do the same arithmetic as the optimized code, otherwise the speedup measures two different algorithms. And on a CPU with two kinds of core, the number of threads is the wrong yardstick for the speedup; what the cores can actually deliver is the right one.

7. Conclusions

I implemented mean shift for colour segmentation in C++ with OpenMP and SIMD. The parallel version needs no synchronization apart from reductions, and it gives the same clusters and labels as the sequential one, with modes within 0.08 on the 0–255 scale.

The fastest configuration is `par` with the SoA layout, 8 threads and `dynamic,16`. On the 128×128 image with $h = 12$ it takes

1.51 s against 13.15 s for the scalar baseline, a speedup of $8.72\times$ ($1.84\times$ from SIMD, $4.75\times$ from threads), and up to $15.2\times$ with large bandwidths. Threads scale almost linearly on the 4 performance cores and much less on the efficiency cores.

Several things could make it faster. The vector loop could skip far neighbours too, for example by sorting or bucketing the points by colour, or with a k-d tree. The blurring variant of mean shift converges in fewer iterations. Adding pixel coordinates to the colours (joint spatial-range mean shift) would give better segmentations, and a GPU version would make full-resolution images practical.

References

[1] Parallel programming course slides. University of Florence. 1

[2] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967. 8

[3] P. Arbeláez, M. Maire, C. Fowlkes, and J. Malik. Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916, 2011. 4

Appendix

1. **Computer Vision components:** the project implements mean shift, a non-parametric feature-space analysis technique used for mode seeking and image segmentation. The algorithm performs kernel density estimation and gradient ascent on pixel colour coordinates (RGB).

2. **Contributions:**

   (a) **Ideas:** noticed that the points are independent and that the uneven number of iterations calls for dynamic scheduling.

   (b) **Formulations:** rewrote the Gaussian weighting with a branch-free cutoff and an inline exponential so that it could be vectorized.

   (c) **Software/Coding:** wrote the whole C++ code (sequential baseline, SIMD, OpenMP and layout variants), the benchmark and the Python scripts for preprocessing and plots.

   (d) **Experiments:** designed the measurement protocol (warm-up, repetitions, confidence intervals, cool-down) and the experiments on threads, bandwidth, schedule and layout on BSDS500 images.

   (e) **Analysis:** analysed speedup, efficiency and bottlenecks.

3. **Authorship:** I confirm that I am the sole author of this writeup and the associated code.